

T1.4.D follow-up — estado fresco + server-only

Stripe como fuente más fresca · server-only en internal-bridge

Owner: Celestino · 2026-05-03 · Branch `pr/t1.4.d-dashboard-data` · Commit follow-up `93b9401`

Status: Ajustes pedidos por Dona Control aplicados sobre el PR de T1.4.D. Branch repusheada. tsc 0 errores · build OK · `/api/dashboard-data` sigue como `f`. Lint 16 = baseline.

1. Cambios aplicados

Archivo	Δ líneas	Cambio
landing/app/api/dashboard-data/route.ts	+13 / -0	Merge ahora deriva estado y puede_cancelar desde Stripe overrides cu
landing/lib/internal-bridge.ts	+6 / -0	Agregado <code><i>import "server-only"</i></code> + comentario explicando que evita
TOTAL follow-up	+19 / -0	2 archivos

2. Diff resumido

a) Merge actualizado en route.ts

Antes:

```
// 5. Merge: datos del backend + Stripe + email de sesión.
const merged = {
  ...data,
  usuario: { ...data.usuario, email: sessionEmail },
  suscripcion: {
    ...data.suscripcion,
    current_period_end: stripeOverrides.current_period_end,
    cancel_at_period_end: stripeOverrides.cancel_at_period_end,
  },
};
```

Después:

```
// 5. Merge: datos del backend + Stripe + email de sesión.
// Para `estado` y `puede_cancelar` Stripe es fuente más fresca: si
// el webhook al backend está retrasado, Stripe ya sabe si la sub fue
// cancelada o quedó past_due. Si Stripe no está disponible
// (defaults null), usamos lo que diga el backend.
const data: UsuarioResumen = backendRes.data;
const estadoFinal =
  stripeOverrides.status_stripe ?? data.suscripcion.estado;
const puedeCancelar =
  estadoFinal === "active" && !stripeOverrides.cancel_at_period_end;

const merged = {
  ...data,
  usuario: { ...data.usuario, email: sessionEmail },
  suscripcion: {
    ...data.suscripcion,
    estado: estadoFinal,
    current_period_end: stripeOverrides.current_period_end,
    cancel_at_period_end: stripeOverrides.cancel_at_period_end,
  },
  resumen: {
```

```

    ...data.resumen,
    puede_cancelar: puedeCancelar,
  },
};

```

b) server-only en internal-bridge.ts

```

// `server-only` evita que cualquier Client Component o código de
// cliente lo importe accidentalmente: si pasa, el build de Next
// falla con un error claro. Estas funciones leen
// INTERNAL_BRIDGE_SECRET y nunca deben correr en el browser.
import "server-only";

import crypto from "node:crypto";

```

3. Lógica de estado y puede_cancelar

Caso	estado_final	puede_cancelar
Stripe active + cancel_at_period_end=false	active	true
Stripe active + cancel_at_period_end=true (cancelación pendiente)	active	false
Stripe canceled (backend aún no procesó webhook)	canceled	false
Stripe past_due	past_due	false
Stripe falla / no configurado (status_stripe=null)	del backend	active && !backend_cancel_period (default false)

Implicación: si el backend dice 'active' pero Stripe ya marcó 'canceled' (webhook retrasado), el dashboard mostrará 'canceled' y el botón de cancelar quedará deshabilitado. Coherente con realidad.

4. Comandos ejecutados

Comando	Resultado
npx tsc --noEmit	0 errores
npm run lint	16 problems · todos preexistentes · 0 nuevos
npm run build	✓ Compiled · 16/16 static pages · /api/dashboard-data sigue como f

Confirmación de server-rendered

```

Route (app)
■ f /api/dashboard-data      ← sigue como f (dynamic)
■ f /api/auth/[...nextauth]
■ f /api/cancel-subscription
■ f /api/checkout
■ f /api/webhook
■ f /api/whatsapp-webhook
■ f /dashboard
■ ... 9 rutas estáticas

```

5. Riesgos del cambio

- **Stripe SDK falla por rate-limit / red:** caemos al backend para estado y puede_cancelar. El usuario ve datos un poco más viejos pero coherentes con la última info que entró por webhook. Aceptable.

- **Backend reporta 'active' pero Stripe está canceled:** ahora respetamos Stripe — UX coherente.
- **Backend reporta 'canceled' pero Stripe no:** caso raro (backend marcaría canceled solo tras procesar webhook `subscription.deleted`). Si pasa, ahora el dashboard muestra el estado de Stripe (probablemente active) — podría confundir un instante. Mitigación: ese caso es muy improbable en práctica.
- **server-only no rompe build existente:** verificado · el route handler de webhook que lo importa también es server-side.

6. Próximo paso

Branch **pr/t1.4.d-dashboard-data** ya en remoto con los 2 commits:

- **732f247** feat(landing): proxy `/api/dashboard-data` server-side (T1.4.D)
- **93b9401** fix(landing): estado fresco de Stripe + server-only en bridge

Esperando re-review de Dona Control. Si aprueba, mergear vía GitHub UI:
<https://github.com/celestinojbm/Dona-agent/compare/main...pr/t1.4.d-dashboard-data>

Después: T1.4.E (UI dashboard que consume `/api/dashboard-data`) y T1.4.F (cancelación segura con `cancel_at_period_end`).